

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

André Thiago Pfleger
Arthur de Lara Machado
Caio Ferreira Cardoso
Felipe Fagundes Pacheco

Título: Goal Masters

Florianópolis

2023

André Thiago Pfleger

Arthur de Lara Machado

Caio Ferreira Cardoso

Felipe Fagundes Pacheco

Trabalho final submetido à disciplina de Programação Orientada à Objetos II do curso de Ciências da Computação, campus Trindade da Universidade Federal de Santa Catarina como requisito parcial para a obtenção da nota final da disciplina.

Orientadores: Prof. Jônata Tyska Carvalho, Dr. e Bobiquins Estêvão de Mello, Dr.

RESUMO

O presente trabalho foi realizado com o objetivo de aplicar os conhecimentos adquiridos ao longo do curso, tais quais o paradigma de programação orientada a objetos, boas práticas de programação, produção de interfaces gráficas, etc., bem como exercitar trabalho em equipe. Tal trabalho foi realizado utilizando a linguagem de programação Python por meio de suas bibliotecas existentes, a fim de desenvolver um jogo de futebol 2D baseado no conhecido jogo Soccer Heads, onde o objetivo é competir com um amigo por diversão.

Palavras-chave: Goal Masters; Jogo; Pygame, POOII; Python.

ABSTRACT

The current work was made with the objective of applying the knowledge acquired through the course, such as oriented object programming paradigm, good programming practices, graphical interfaces production, etc., as well as exercising teamwork. This assignment was made using the programming language Python and its existing libraries, in order to develop a 2D game based on the known soccer game called Soccer Heads, in which the main goal is to compete with a friend for fun.

Keywords: Goal Masters;Game; Pygame; POOII; Python.

SUMÁRIO

1	**INTRODUÇÃO**	**8**
1.1	TEMA	8
1.2	TECNOLOGIAS ESCOLHIDAS	8
1.3	METODOLOGIA DE DESENVOLVIMENTO	8
1.4	DIVISÃO DE TAREFAS	9
2	**PROPOSTA DO JOGO**	**10**
2.1	JOGADOR E ADVERSÁRIO	10
2.2	MECANISMOS	10
2.3	COLETÁVEIS	10
2.4	CENÁRIOS	11
2.5	REGRAS	11
2.6	OBJETIVO	11
2.7	CONDIÇÕES DE VITÓRIA, EMPATE E DERROTA	11
3	**DESENVOLVIMENTO**	**12**
3.1	INTEGRAÇÃO COM PYGAME	12
3.2	CLASSE MATCH	13
3.3	OBJETOS DE JOGO	14
3.4	COLISÕES	14
3.4.1	POR QUE NÃO SPRITES?	15
3.4.1	LÓGICA DE PROCESSAMENTO DOS MOVIMENTOS	15
4	**MAIORES DIFICULDADES**	**17**
5	**CONCLUSÃO**	**17**
	REFERÊNCIAS	**19**

1 INTRODUÇÃO

O presente trabalho foi realizado sob um viés educativo, a fim de exercitar as técnicas e conhecimentos desenvolvidos ao longo do primeiro semestre letivo de 2023 na disciplina de Programação Orientada a Objetos II da UFSC campus Trindade, do curso de Ciências da Computação. O objetivo deste trabalho foi criar um jogo de nossa escolha na linguagem de programação Python.

1.1 TEMA

O jogo escolhido foi um jogo competitivo baseado no famoso jogo “Soccer Heads”, que é um conceito de jogo onde os personagens são pequenos, com grandes cabeças e, em uma plataforma 2D, devem tentar marcar gol no seu adversário. Porém, toda a *gameplay* é afetada por um conjunto de mecânicas que influenciam a jogabilidade como *buffs* ou *debuffs* (habilidades, obstáculos, efeitos positivos ou negativos), físicas surrealistas onde os personagens não se comportam como os corpos do mundo real, etc.

1.2 TECNOLOGIAS ESCOLHIDAS

A linguagem de programação do projeto é Python, pois é a linguagem na qual foram exercitados os conceitos de programação orientada a objetos. A partir daí decidimos utilizar a biblioteca Pygame para o desenvolvimento do jogo, pois é uma das mais renomadas e conhecidas em Python. Apesar de suas limitações em termos de processamento, é muito utilizada para jogos simples, réplicas de jogos retrô e estudos de programação. Contudo, ela possui uma limitação muito grande no que diz respeito à geração de UIs (*User interfaces*), sendo estas muito verbosas para implementar. Com isso em vista, foi decidido fazer uso de outra biblioteca chamada PySimpleGUI, que facilita e otimiza a criação de interfaces gráficas com o usuário em Python.

1.3 METODOLOGIA DE DESENVOLVIMENTO

O projeto foi desenvolvido levando em consideração as recomendações das bibliotecas utilizadas, dos professores orientadores e do conteúdo da disciplina, tal que chegamos no seguinte conjunto de fases de desenvolvimento descritos abaixo:

1 - Ideia do tema do jogo;

2 - Idealização e construção do diagrama de classes;

3 - Verificação e correção do diagrama;

4 - Desenvolvimento do código:

4.1 - Como adequar o nosso diagrama de classes para que funcione com Pygame;

4.2 - Implementação das classes do diagrama e produção do MVP;

4.3 - Desenvolvimento do modelo completo do jogo com todas as *features* inicialmente idealizadas disponíveis;

4.4 - Correção de bugs.

1.4 DIVISÃO DE TAREFAS

As tarefas foram divididas de forma igualitária e de acordo com as preferências de cada integrante do grupo. O código foi compartilhado por meio do Github e através de conversas buscamos discutir e resolver novas demandas de projeto conforme foram aparecendo.

2 PROPOSTA DE JOGO

2.1 JOGADOR E ADVERSÁRIO

Ambos, jogador e adversário, são jogadores reais que podemos apelidar de jogadores 1 e 2. Ao entrar na partida os personagens de cada competidor poderão ser escolhidos de maneira pré-definida, ou seja, o jogador não poderá editá-los ou criar novos personagens, apenas usar os existentes.

2.2 MECANISMOS

Dentro dos mecanismos temos dois tipos diferentes: o mecanismo de jogo consiste em como o jogador iniciará uma partida e as opções que lhe são disponíveis; já o mecanismo de controle é como os jogadores serão controlados.

No primeiro caso, será apresentado um menu com um botão de "*Start*". Ao clicar nele, o jogador irá ver o menu de partida que consiste em escolher os personagens com quem irá jogar e selecionar o tempo de partida, após isso, o jogo começa.

O segundo mecanismo ocorre após o início da partida, nele o jogador 1 controlará o seu personagem pelas teclas "w", "a", "s", "d" e chutará na tecla "space". Já o jogador 2, pelas setas do teclado e chutará na tecla "p".

Um mecanismo que também acontece durante a partida é o movimento da bola, que apesar de não ser controlada diretamente pelo jogador, é influenciada pelos movimentos discutidos no mecanismo anterior, ou seja, quando o jogador tocar na bola ela será impulsionada na direção do movimento do jogador.

2.3 COLETÁVEIS

O jogo terá um sistema de coletáveis, onde serão aplicados buffs ou debuffs dentro da partida. Primeiramente serão buffs e debuffs que alteram os objetos de jogo, como jogador, trave e bola, então por exemplo terá um buff que irá diminuir o tamanho da sua trave/gol, dificultando ao seu adversário de marcar gol, ou um debuff que quando a bola passar por ele, o último jogador que tocou na bola será congelado por alguns segundos. Os coletáveis surgem de forma aleatória no decorrer da partida, de forma que deixe o jogo mais dinâmico.

2.4 CENÁRIOS

Haverá um cenário padrão que é em um estádio e um cenário no deserto. Os cenários não influenciam na forma como o jogo é jogado, apenas na estética e no som que o jogo produz.

2.5 REGRAS DE JOGO

Um ponto, ou gol, é marcado no momento que a bola atravessa a trave adversária. O acúmulo de tais pontos é limitado sendo o limite 7 pontos.

2.6 OBJETIVO

O objetivo de cada partida é vencer o adversário da maneira previamente estabelecida, marcando o número máximo de pontos no tempo limite estabelecido.

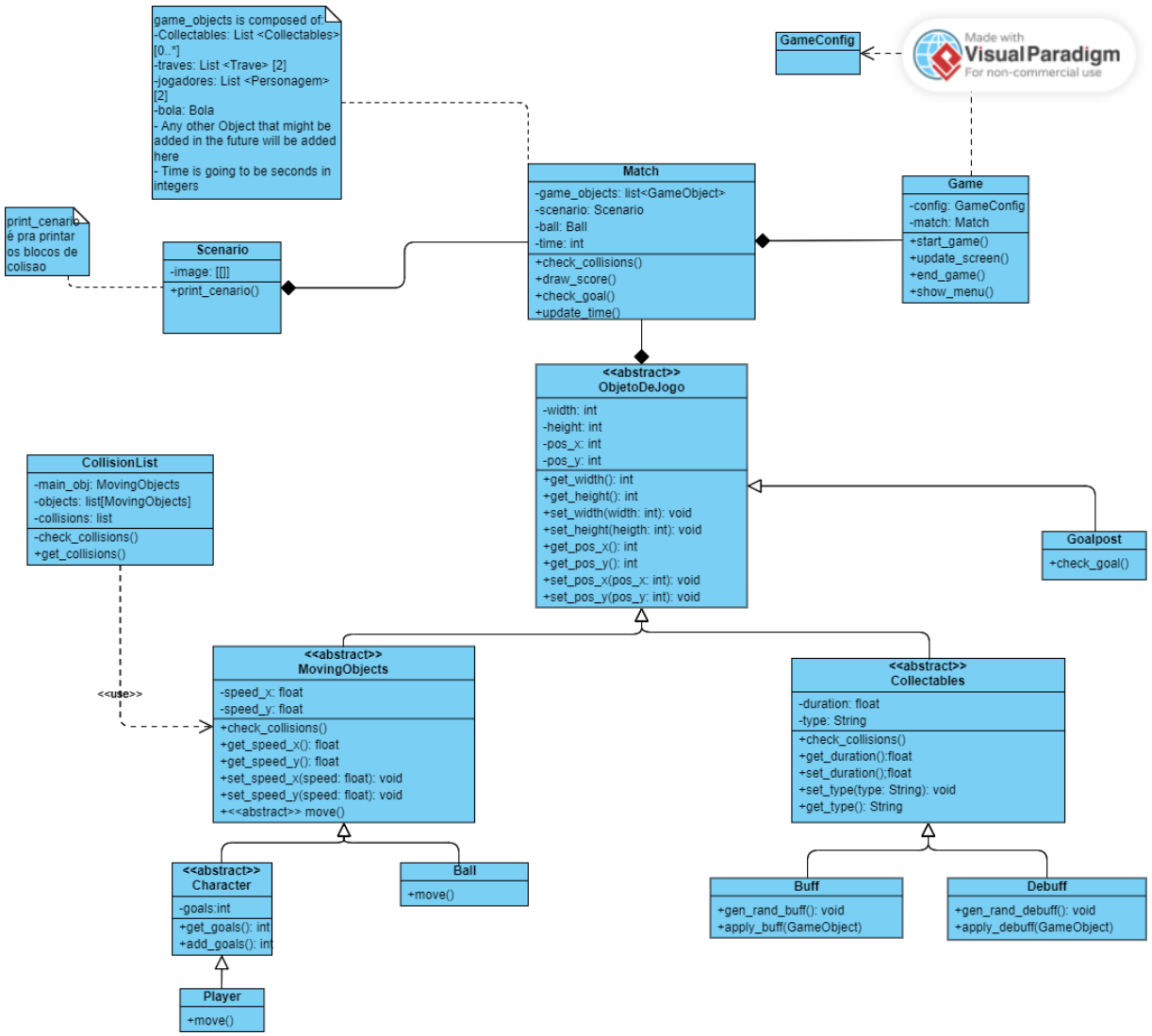
2.7 CONDIÇÕES DE VITÓRIA, EMPATE E DERROTA

O jogador vitorioso, será aquele que ao final do tempo estabelecido, tiver feito mais gols. Em caso de empate, será decidido o vencedor, com o critério de quem fizer o próximo gol (Gol de Ouro), nesta etapa, o tempo ficará parado/não contabilizando. A derrota será de quem sofreu com as etapas anteriores, ou seja, tomou mais gols durante o decorrer do tempo e no caso de empate, será aquele que sofreu o Gol de Ouro.

3 DESENVOLVIMENTO

A seguir temos um diagrama sob o qual baseamos nosso desenvolvimento, e que usaremos para explicar em mais detalhes cada uma das partes do projeto.

Figura 1 - Diagrama de classes - Goal Masters



Autores: PFLUGER, André Thiago; MACHADO, Arthur de Lara; CARDOSO, Caio Ferreira; PACHECO, Felipe Fagundes

3.1 INTEGRAÇÃO COM PYGAME

Primeiramente foi necessário entender como a biblioteca do Pygame funcionava e como permitir que o diagrama pudesse ser adaptado às necessidades da tecnologia. Para tal, fora decidido que a classe *Game* seria a responsável por

gerenciar a UI do jogo com o sistema de partidas. Cada partida (classe *Match*) teria um início e fim e gerenciaria em si os objetos existentes, regras de jogo, etc.

Com isso em vista, a classe *Game* ficou responsável pelo *loop* principal do jogo, que é o laço principal que permite a geração dos frames. Contudo, ela não afeta em nada na jogabilidade, pois quem gerencia isso é a classe *Match*; assim sendo, era necessário que ambas as classes conversassem entre si, de tal modo que *Game* não só pudesse iniciar uma partida, mas dar a *Match* o poder de gerenciar o estado dos objetos de jogo. Por isso, o laço principal supracitado contém algumas funções padrões, na seguinte ordem:

- **handle_events():** responsável por detectar os eventos como teclas pressionadas e mensagens de componentes de jogo e passá-los adiante para a classe *Match*;
- **process_input():** responsável por passar os eventos detectados em handle_events() para o processamento de dados de entrada da classe *Match*;
- **draw_frame():** responsável por chamar a função que desenha o estado atual da partida;
- **render():** responsável por colocar na tela o desenho feito por draw_frame().

3.2 CLASSE MATCH

A classe *Match* é quem detém a lógica de jogo. Todas as informações necessárias sobre o Pygame lhe são passadas pela classe *Game*, como demonstrado anteriormente, assim, mesmo que um novo modo de jogo seja implementado, tudo o que ele precisa é conter uma nova lógica de jogo, tal que as funções básicas citadas no capítulo 3.1 deste trabalho se mantenham as mesmas.

Vale também ressaltar que a classe *Match* só é responsável por gerenciar a forma como a partida ocorre, mas não altera o estado dos objetos nela existentes. O que acontece é que a classe em questão detém o equivalente à “informações globais” da partida, se assim pudermos chamar, como os objetos existentes, tempo de jogo e placar. Entretanto, as regras de negócio de cada um dos objetos, bem como seus estados atuais são definidos por eles mesmos, lhes concedendo independência, diminuição no acoplamento do código e maior reusabilidade, a partir do momento que eles podem ser programados de qualquer maneira, contanto que respeitem as informações concedidas pela classe *Match*.

3.3 OBJETOS DE JOGO

Os objetos de jogo são todo e qualquer objeto que possa ser colocado dentro da partida. Porém alguns são estáticos e outros móveis.

Os objetos de jogo móveis são chamados de *MovingObjects* e herdam dessa classe abstrata. Aqui são definidas questões padrões necessárias para o movimento de cada objeto móvel. Exemplos desses objetos são a bola e o jogador.

Os objetos estáticos são objetos de jogo normais, que simplesmente não herdam de *MovingObjects* como goleiras, chão, etc.

Todo objeto de jogo gerencia as interações dele com o ambiente, sendo o ambiente, o cenário e até mesmo outros objetos de jogo. Para isso, existe uma lógica de colisão própria para cada objeto criado, que leva em consideração o objeto com o qual houve a colisão e que comportamento ele deve ter a partir disso. Com exceção de um caso específico da bola e do jogador (que será comentado mais a frente) todos os objetos de jogo apenas alteram o seu próprio estado. Assim, mantemos uma maior coesão e fácil manutenção do código.

E quando um objeto deseja comunicar-se com o estado da partida *Match* ou afetar algum outro objeto de jogo, sem que haja colisão ou algo que o valha? Para isso, utilizamos uma forma de resolução parecida com o padrão de projeto Observer. Fora tirado proveito do método **get_events()** do Pygame, que permite com que a cada *frame* uma série de eventos sejam detectados, como uma tecla pressionada, por exemplo. Ademais, ele permite que enviemos eventos customizados, chamados de *User Events*, disponibilizando uma forma de comunicação parecida com a do padrão Observer, onde um objeto pode sinalizar um evento que pode ser escutado por todos os objetos interessados. Aqueles que tiverem interesse em “ouvir” esse sinal, podem então fazê-lo e tomar as medidas desejadas a partir daí.

3.4 COLISÕES

O Pygame tem sua própria maneira de lidar com colisões, mas trabalha muito mal com múltiplos objetos que se movem de maneira simultânea e indeterminada, como múltiplos jogadores, por exemplo, onde é impossível definir quais serão seus próximos movimentos e onde irão colidir. Um dos fatores para isso é a dificuldade de se entender o eixo no qual a colisão ocorreu. Esses problemas e alguns outros serão discutidos nesta sessão.

3.4.1 POR QUE NÃO SPRITES?

O Pygame utiliza uma classe chamada *Sprite*, que disponibiliza uma série de funções de colisão. Contudo, essas funções não só não pareciam nos ser tão úteis, acabando por exigir trabalho extra, como se mostraram não valendo a pena serem implementadas devido ao custo de refatoramento do projeto, que seria alto. Isto porque o projeto final foi construído em cima do MVP e este, por sua vez, foi projeto visando colisões entre retângulos (Pygame *Rects*). A partir disso, a classe partida possui uma lista de objetos de jogo que detém as informações desses *Rects*, de tal maneira que seria necessário refatorar boa parte da lógica dos *GameObjects* para aplicar a lógica pronta de *Sprites,* quando, na verdade, a lógica de *Sprites* faz uso dos *Rects* por baixo dos panos para calcular suas colisões. Era mais fácil simplesmente mantermos a lógica de colisão do MVP e ao desenhar os nossos objetos de jogo na função **draw()** de cada um, incluir uma pequena lógica que permite desenhar uma imagem sob o seu retângulo.

Daí surgiu a classe ***CollisionList*** utilizada pelos *MovingObjects*. Ela permite identificar todos os objetos que colidiram com um determinado objeto A no presente *frame*, e disponibiliza um array com todas as colisões para que A saiba como proceder a partir de cada uma delas, considerando os aspectos necessários, como tipo do objeto com que colidiu, parâmetros, etc.

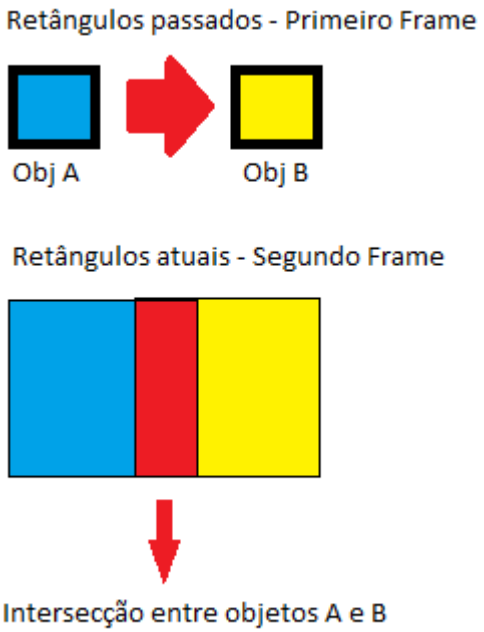
3.4.2 LÓGICA DE PROCESSAMENTO DOS MOVIMENTOS

A lógica de processamento dos movimentos poderia ser uma sessão independente, mas como esta se encontra diretamente relacionada com as colisões, sendo praticamente um único processo, será citada aqui em conjunto com mais detalhamentos sobre o formato das colisões.

Se movermos um *MovingObjects* em mais de um eixo de maneira simultânea, podemos acabar com problemas para a colisão, ocasionando *Overlapping* (que nada mais é do que quando um objeto se sobrepõe à outro) visto que pode não ser possível identificar como a colisão ocorreu (o eixo e direção da qual o objeto veio) e o que fazer em seguida. Portanto, separamos os movimentos por eixo individualmente, assim garantindo que quando houver uma colisão, saberemos o eixo em que ocorreu. Quanto à direção, bastaria verificar a velocidade do objeto; o

sinal deve indicar direita, esquerda, cima ou baixo, a depender do eixo em questão. O problema com essa resolução é que em casos muito específicos como experienciamos ao longo do desenvolvimento do Goal Masters, esse tipo de resolução pode trazer problemas ao resultado final e permitir erros múltiplos objetos colidem entre si ao mesmo tempo, até mesmo porque alguns deles têm suas velocidades diretamente influenciadas por estas colisões. Desta maneira, uma nova abordagem se fez necessária. A ideia é que verifiquemos um retângulo passado (posição do objeto no *frame* anterior) e o retângulo atual. Assim, se o objeto A colide com o objeto B, onde o lado direto do retângulo passado do objeto A era menor que o lado esquerdo do retângulo passado do objeto B e o lado direito retângulo atual do objeto A é maior que o lado esquerdo do retângulo atual do objeto B, verificamos uma colisão à direita do objeto A e à esquerda do objeto B, como demonstrado na figura 2 abaixo.

Figura 2 - Intersecção entre objetos



Agora é fácil entender como a colisão ocorreu e como devemos lidar com ela. No início de cada *frame* o objeto é movido primeiro no eixo ‘x’, nas direções direita ou esquerda a depender de sua velocidade. Então verificamos se houve colisão da forma descrita acima e lidamos com a mesma. Para correção do *Overlapping* realizamos um pequeno ajuste na posição final do nosso objeto, de maneira imperceptível para o usuário final. Após, também efetuamos o mesmo processo, só que agora para o eixo ‘y’, onde a velocidade indica para cima ou para baixo. Todo este processo acontece antes do objeto ser desenhado na tela, permitindo que o *Overlapping* nunca ocorra. Na prática os objetos de jogo irão se sobrepor, e é isso que nos permite identificar colisões, porém essas sobreposições são visualmente corrigidas antes da imagem ir para a tela.

4 MAIORES DIFICULDADES

As maiores dificuldades do jogo foram principalmente as colisões. Alguns objetos deram mais trabalho para a implementação, como os pés dos jogadores. Entretanto, tudo se resume à dificuldade que tivemos para lidar com o sistema de colisões e encontrar uma solução óptima. Problemas como Tunneling e Overlapping ocorriam em demasia, mas, felizmente, assim que foi encontrado uma lógica que melhor coubesse no projeto, não foram necessários muitos refatoramentos, graças à estrutura provida pelo diagrama de classes que serviu bem ao jogo de maneira geral.

5 CONCLUSÃO

Faz-se imprescindível o uso de boas práticas de programação, tais quais as descritas nos livros do ‘Uncle Bob’ e que nos foram apresentadas na disciplina de Programação Orientada à Objetos II. Neste projeto pudemos conhecer mais sobre como trabalhar em equipe como desenvolvedores e entender a importância de boas estruturas e práticas de projeto, sem as quais o jogo provavelmente teria rapidamente se tornado um aglomerado de bugs, mesmo que atualmente não se encontre em um estado perfeito. O ponto é justamente que a dificuldade para se lidar com os problemas atuais são pequenas se comparado ao que seriam sem tais técnicas de implementação. A organização do projeto permitiu que ele crescesse em

termos de *features* disponíveis e passasse do MVP para a versão final sem a necessidade de reestruturá-lo do zero.

REFERÊNCIAS

Radivojevic, Igor. Soccer Players pixel art [Pixel art]. Disponível em: https://dribbble.com/shots/3330157-Soccer-Players-pixel-art. Acesso em: 2 de julho de 2023.

ClearCode. Como programar em Python: Tutorial para iniciantes [Vídeo]. Disponível em: https://www.youtube.com/watch?v=hDu8mcAlY4E&ab_channel=ClearCode. Acesso em: 2 de julho de 2023.

Patterns Game Programming. Discover Python and Patterns: 8. Game Loop Pattern. Disponível em: https://www.patternsgameprog.com/discover-python-and-patterns-8-game-loop-pattern. Acesso em: 2 de julho de 2023.

Game Programming Patterns. Observer. Disponível em: https://gameprogrammingpatterns.com/observer.html. Acesso em: 2 de julho de 2023.